

File System Project

Team Name: AAXF

Team Members: Aidan Bayer-Calvert (922119403), Arric Sekhon (923090490), Fabian Camarena Veliz (924100096), and Subhan Khan (921938807)

Github link:

<https://github.com/CSC415-2025-Spring/csc415-filesystem-FlavyFabo>

Description:

This CSC-415 group project implements a three-phase, file system entirely in C. In Phase 1 we format a volume by writing a partition header, creating a Volume Control Block (VCB), and initializing a free-space bitmap. In Phase 2, we built directory support (fs_mkdir, fs_rmdir, fs_opendir/fs_readdir/fs_closedir, fs_getcwd/fs_setcwd, fs_isFile/fs_isDir, fs_delete). In Phase 3 we layer on file I/O (b_open, b_read, b_write, b_seek, b_close) plus metadata via fs_stat. The simple shell (fsshell.c) enables use of UNIX-style commands such as ls, cp, mv, rm, touch, cat, cd, pwd, history, and copy-to/from-Linux bridges.

Plan/Approach

(The plan/approach for each phase and changes made)

Milestone 1: File System Initialization

Our first goal was to set up the foundation of the file system. This included initializing the Volume Control Block (VCB), setting up the bitmap-based free space manager, and creating the root directory. We started this phase around **April 2**, right after getting our milestone instructions.

- We all met as a team and discussed how we would divide the tasks. We created a GitHub repository and made sure everyone could pull and push code and test the sample builds.
- **Arrie Sekhon** worked on the VCB setup and the main initialization logic in fsinit.c. He handled the format detection, memory allocation for VCB, and saving metadata like blockSize, totalBlocks, and rootDirStart.
- **Aidan Bayer-Calvert** focused on the free space bitmap. He calculated how many bytes and blocks the bitmap would need and helped write the code that initialized and flushed it to disk.
- **Fabian Camarena Veliz** helped test the initial setup and assisted in creating the root Directory. He also helped debug bitmap size calculations.
- **Subhan Khan** worked on the initial version of the FreeSpaceManager structure and its allocateBlocks and releaseBlocks methods. He also reviewed how the bitmap should be laid out.

We had issues with memory alignment and incorrect block calculations early on. We fixed this by updating our bitmap math and rechecking how many blocks the bitmap would span. Once we fixed that and made sure the bitmap and root directory were flushed properly, the base file system worked.

Milestone 2: Directory Creation and Navigation

For Milestone 2, we needed to make directory commands work such as **mkdir**, **cd**, and listing directories. We also had to make sure directories could be navigated with paths like **../folder**.

- We had a team meeting around **April 10–12** to plan the structure of directories. We assigned key functions to each person and discussed how to handle `fdDir` and `directory_entry`.
- **Arric Sekhon** worked on all the core path navigation logic. He built `fs_setcwd`, `fs_getcwd`, `fs_isFile`, `fs_isDir`, and `fs_mkdir`. He focused on making sure directories were being correctly created and that path resolution worked.
- **Aidan Bayer-Calvert** focused on deletion functions (`fs_rmdir`, `fs_delete`) and the `fs_stat` function. He also cleaned up formatting and helped organize directory traversal.
- **Fabian Camarena Veliz** helped with directory testing and implemented listing functionality. He worked on `fs_readdir` and verified that entries like `"."` and `".."` were working correctly.
- **Subhan Khan** worked on testing and debugging file and directory commands. He checked how directory structures were being read from disk and helped test edge cases.

We faced a serious issue where directories would be created but `cd` wouldn't work on them. The problem was that we didn't flush metadata properly using `LBAwrite`. We added debug prints and eventually fixed it by ensuring everything was written to disk after creation.

Milestone 3: File I/O and Commands (`b_io.c`)

This milestone was focused on reading and writing files, and implementing the commands like `cp`, `cat`, and `cp2fs`. We had to make sure `b_open`, `b_write`, `b_read`, and `b_close` worked correctly.

- Around **April 20–22**, we met again to split up the work for file operations and command-line tools.
- **Arric Sekhon** implemented `b_open`. He made sure files could be opened and that new files got entries in directories. He also worked on allocating blocks when needed and checking flags like `O_CREAT` and `O_TRUNC`.
- **Aidan Bayer-Calvert** implemented `b_write`, `b_read`, and the command-line command `cmd_rm`. He added checks to ensure correct data writing and flushing file metadata back into the directory entries.
- **Fabian Camarena Veliz** helped test all file operations. He tried different file sizes and helped spot bugs in how data was written when crossing buffer boundaries.
- **Subhan Khan** implemented the `cp`, `cp2fs`, and `cp2l` commands. He tested them with various files and ensured that files copied into and out of the system worked correctly. He also added comments to the free space and command code.

We ran into problems like files appearing in listings (`ls -l`) but not being readable using `cat`. After debugging, we realized the issue was missing reinitialization of the bitmap when the system was already formatted. We fixed this by calling `initFreeSpaceManager` even in the formatted state.

We also ensured all directory updates were flushed.

We had regular meetings every week between **April 2 and May 4**. Early in the project, we met more frequently (2–3 times a week), and during the final week, we often did pair debugging and testing over video call or in person.

A list of exactly which shell commands work and don't work or limitations:

ls - working
cp - working
mv - working
md - working
rm - working (limitation is directory has to be empty to remove. This was an intentional design choice, but could be considered a limitation).
touch - working
cat - working
cp2l - working
cp2fs - working
cd - working
pwd - working
history - working
help - working

Issues and Resolutions:

In the early stages of our implementation, we encountered a problem where directories could be created using `fs_mkdir`, but they could not be opened or navigated into afterward. When attempting to `cd` into the newly created directory, the system returned an error indicating that the directory did not exist. This was confusing because the directory entry was visible but inaccessible. We addressed this by adding debug print statements to trace execution, carefully examining the path normalization logic and how directory structures were persisted. It turned out that directory metadata was not correctly flushed to disk, and in some cases, directory names were not stored correctly. After fixing these issues and ensuring consistent reading/writing of directory structures using `LBWrite` and `LBRead`, navigation worked as expected.

When using `cd ..` to move up one directory level, we observed erratic behavior where the current working directory would either not change or return unexpected paths. Since our file system in Milestone 2 only supports directories directly under root, attempting to access a parent directory above root didn't make sense and resulted in undefined behavior. We improved `fs_opendir` and `fs_setcwd` to detect `".."` and treat it as a request to return to the root directory (`/`). This special case handling made our system more robust and prevented it from navigating into unsupported directory levels.

Sometimes during file or directory creation, the system failed to allocate a proper data block, causing operations to fail unpredictably. These errors were difficult to trace and led to corrupted

or missing files. We discovered that although blocks were being marked as used in the bitmap during allocation, the changes were not being flushed to disk. As a result, the system thought those blocks were still available during subsequent operations. We updated the allocation process in `createDir` and related functions to always write the updated bitmap to disk using `LBAwrite` immediately after any change. This ensured that the in-memory and on-disk states remained consistent.

We ran into a serious bug where blocks that should have been reserved for system metadata such as the Volume Control Block (VCB), bitmap, and root directory were being reallocated for files or directories. This led to overwriting critical data and system instability. The free space manager was unaware of which blocks were reserved because we hadn't marked those blocks as "in use" during initial formatting. In the `initFreeSpaceManager` function, we explicitly marked all blocks used by the VCB, bitmap, and root directory as occupied. This prevented the free space system from allocating them again and protected essential file system structures.

Correctly calculating the size of the bitmap in both bytes and blocks proved to be tricky. Early attempts often resulted in allocating too little memory, which led to memory overflows or bitmap corruption.

We corrected the formulas used to calculate bitmap size. For bytes:

`bitmapBytes = (totalBlocks + 7) / 8;`

To

`bitmapBlocks = (bitmapBytes + blockSize - 1) / blockSize;`

These formulas ensured correct rounding and alignment, allowing our bitmap to scale accurately with the number of blocks.

During testing, we noticed that the file system sometimes reformatted the volume even when it had already been initialized and formatted. This caused data loss and unnecessary reinitialization. We added a proper check for the VCB signature (`g_vcb->signature != VCB_SIGNATURE`) to confirm whether the volume was already formatted. After a successful format, we wrote the signature to the VCB. This prevented unnecessary reformatting and ensured persistent volume state.

After formatting the volume, we sometimes couldn't list or access the root directory. Commands like `ls` or `cd` returned empty results or errors.

We realized that either the root directory wasn't written to disk using `writeDir`, or the starting block location wasn't set properly in the VCB.

We made sure to write the root directory entries to disk right after formatting and correctly assigned `g_vcb->rootDirStart`. This guaranteed the root could be accessed immediately upon file system initialization.

As we reinitialized the file system during development, we noticed memory leaks and leftover data persisting across runs. Inconsistent behavior and crashing during repeated mounts were signs of improper memory management. We updated the shutdown and reinitialization routines to ensure that memory allocated for the root directory and the free space bitmap were properly freed using `free()`. This eliminated memory leaks and ensured a clean restart environment every time the volume was mounted.

One issue we faced was with the `b_write()` function not updating the file size on disk. Even though data was written to the file, the size shown in `ls -l` remained zero. This happened because we only updated the in-memory copy of the directory entry and forgot to persist those changes back to disk. To fix this, we added a patch in `b_write()` that reloads the directory block, updates the file size field, and rewrites it to disk using `LBWrite`. This way, the actual file size is reflected correctly the next time the system reads directory metadata.

```
Prompt > cd test
Prompt > cp2fs hello.txt
[DEBUG] Writing to testfs_fd = 0
f->fileSize = 0
dirSlot = 3
dirStart = 37
dirBlocks = 31
g_vcb->blockSize = 512
Prompt > ls -l hello.txt
hello.txt is not found
Prompt > ls -l

-          0   hello
-          0   hello.txt
```

We also ran into a problem when testing with `cp2fs`, where files would appear in `ls -l`, but their content wasn't readable using `cat`, or the system would say the file didn't exist. After testing various cases, we found the issue was in the way `b_open()` split the file path and handled relative directories. If the working directory wasn't correctly resolved or opened, it silently failed to find or create files in the correct path. We added debug print statements to verify path splitting and directory handling and ensured we were always opening the correct directory with `fs_opendir()` and creating the file in the right slot.

Another tricky issue occurred when we wrote to a file and then immediately tried to read it back using `cp` or `cat`, but the read returned nothing. After investigation, we found that our internal file pointer (`f->pos`) in the file control block wasn't being reset when the file was re-opened for reading. To fix this, we made sure `b_open()` always sets `f->pos` to 0 unless `O_APPEND` is set. This ensured reading starts from the beginning of the file unless specified otherwise.

While testing block allocation, we encountered cases where `alloc_one_block()` returned `UINT32_MAX`, causing writes to fail silently. This was because our bitmap looked full, even though it had free space. The root cause was that our bitmap in memory was not properly synchronized with the version on disk. We added debug output in `allocateBlocks()` to trace free

block detection, and added bitmap flushing after any allocation using LBAwrite, which made sure changes were consistent across reboots and during concurrent operations.

Lastly, we had an issue where `ls -l hello.txt` worked fine in some directories but failed in others. It turned out that our directory reading logic wasn't correctly skipping over unused directory entries or was misinterpreting entry names due to improper string termination. We made sure every entry name was null-terminated and that `inUse` flags were properly set and checked during directory traversal. This cleared up the inconsistencies when accessing file metadata.

Details of how each of your functions work:

`fs_getcwd`

This function returns the current working directory by copying it into a provided buffer. It ensures the buffer is valid and properly null-terminated, so the user can reliably retrieve their current location in the file system.

`fs_setcwd`

This function changes the current working directory to the path provided. It handles both relative and absolute paths, removes trailing slashes, and treats `".."` as a request to go back to root since only root-level directories are supported. It only updates the path if the destination exists.

`fs_isDir`

This function checks if a given path points to a valid directory. It tries to open the path as a directory and returns 1 if successful, or 0 if it fails, helping confirm whether a location is a folder.

`fs_isFile`

This function checks whether a given path refers to a file. It splits the path into parent directory and file name, opens the directory, and looks for a matching name marked as a regular file. It returns 1 if found, otherwise 0.

`load_root_directory`

This internal helper function reads the root directory from disk into memory. It calculates how many blocks are needed based on the expected number of entries, loads the blocks, and wraps the data in a structure that the rest of the file system can use.

`fs_opendir`

This function opens a directory at a given path. It formats the path, ensures it's valid, and loads the correct blocks from disk. It supports only the root and single-level subdirectories, and it handles special cases like `".."` by redirecting to root.

fs_readdir

This function reads the next valid (in-use) entry from an open directory. It skips over unused slots and fills a structure with the file or directory's name and type, allowing the shell or user to list directory contents one by one.

fs_closedir

This function closes a directory that was opened earlier. It frees the memory that was allocated for reading the directory entries and the structure itself to ensure no memory leaks occur.

fs_mkdir

This function creates a new directory under the root directory. It verifies the name is valid and doesn't already exist, finds an available slot, sets up metadata, updates the bitmap, writes the directory to disk, and adds the new entry into the root so it can be accessed later.

b_init

This function sets up the system by marking all file control blocks (FCBs) as free. It runs during startup to make sure the file system is ready to open files by initializing each slot in the FCB array.

b_getFCB

This function searches through the FCB array to find a free slot (where the buffer is NULL) and returns the index of that slot. If no slots are available, it returns -1 to indicate that all FCBs are in use.

make_absolute_path(char *destination, const char *base, const char *name)

This small helper exists to shield the rest of your code from the details of path concatenation and edge cases around the root directory. When you're in "/" you must avoid producing "//foo" (which some APIs might treat differently), so it special-cases a single-slash base. Otherwise it simply joins base and name with a / between them. By centralizing this logic here—and always forcing the final byte to '\0'—you ensure that every time the file system builds a candidate path (for cd, for lookup, for open, etc.) you won't accidentally overrun buffers or mishandle trailing slashes.

split_path(const char *path, char *parent, char *child)

Almost every file-or-directory operation needs to break a user-supplied path into its parent container and the final "leaf" component (the file or directory name itself). `split_path` implements that split robustly: it first strips any extra trailing / so you don't get empty names, then finds the last slash. If there is none, it returns "." as the parent (current directory); if the only slash is at the front, it returns "/" as the parent. By returning both pieces in clearly sized buffers and

normalizing edge cases (empty string, root, trailing slashes), you can write the rest of your code without re-implementing path parsing everywhere.

fs_rmdir(const char *path)

This function safely removes an empty directory and reclaims its blocks. It first validates the input and ensures you're only operating directly under the root (no multi-level support). It then opens the target directory to verify that only "." and ".." remain, failing with ENOTEMPTY if there are any real entries. Next it re-opens the parent, unlinks the directory entry, writes back the updated directory block, and computes exactly which LBAs held the directory's contents. It loads the on-disk bitmap, clears those bits to mark the blocks free, zeroes out the blocks themselves (to avoid leaving stale data), persists the updated bitmap, updates the in-memory VCB free-block count, and finally cleans up all allocated buffers. That thorough sequence ensures you never leave half-deleted directories or orphaned blocks behind. It only supports removing a directory if there are no files/directories inside. This is because usually when removing you could have the -r (recursive flag) to do so but because this is a simple filesystem, we thought the best approach would be just to have singular removal if it is empty to ensure nothing is accidentally deleted that could be nested in directories.

fs_delete(char *path)

Much like fs_rmdir but for files, fs_delete logically unlinks a single file entry and reclaims its data blocks. After basic validation and ensuring the path points to an actual file, it opens the parent directory, finds and clears the matching entry, writes the directory back to disk, and closes it. It then calculates how many blocks the file occupied, builds a list of those block numbers, and hands that list to the free-space manager to clear the bits in the bitmap. It updates the on-disk bitmap, bumps the VCB's free-block count, and frees its temporary buffers. By centralizing file deletion here, you ensure that every file removal correctly updates both metadata and data allocation.

fs_stat(const char *path, struct fs_stat *buf)

This implements the classic UNIX-style stat() call. It first checks for NULL arguments to prevent crashes, then handles the special case of "/" by simply returning the root directory's block count and zeroed timestamps. For any other path, it uses split_path + fs_opendir to locate the parent, searches for the matching entry by name, and if found fills in the struct fs_stat from that entry's fileSize, accessed_at, modified_at, and created_at fields—computing the block count by rounding up to the next full block. By mirroring the POSIX stat interface, you enable utilities like long-format ls -l (or your own extensions) to be built on top of your file system.

local_split_path

This helper function breaks a full file path into two parts: the parent directory and the file name (leaf). For example, /folder/file.txt becomes /folder and file.txt. This helps with navigating and

organizing the file structure.

open_target_dir

This helper function tries to open the correct directory based on the given path. If the path is absolute (starts with /), it opens it directly. If it's relative, it combines it with the current working directory to get the full path.

find_or_create_in_dir

This function looks for a file with a specific name (leaf) in the given directory. If it finds it, it returns its index. If it doesn't and the O_CREAT flag is set, it creates a new file entry, fills in metadata like name, timestamps, and file type, and writes the updated directory to disk. If creation isn't allowed or no space is available, it returns -1.

b_open(char filename, int flags)

This is one of the most important functions. It prepares a file to be used for reading or writing. It splits the path, opens the parent directory, and finds or creates the file entry. If the file is new and doesn't have storage yet, it allocates its first data block. If O_TRUNC is passed, it resets the file size to 0. Then it fills out a File Control Block with details like file size, position, start block, and allocates a buffer. Finally, it returns a file descriptor (index into fcbArray).

b_write(int fd, char src, int count)

This function writes count bytes from the source buffer src into the file represented by the file descriptor fd. It calculates which block to write to based on the file position. If the block is partially full, it reads the block first (read-modify-write). If the file grows in size due to the write, it updates the directory entry on disk with the new size.

b_read(int fd, char buffer, int count)

This function reads count bytes into buffer from the file. It breaks the process into three parts:

1. Read from the leftover buffer (if any)
2. Read full blocks directly into the buffer (for speed)
3. Read a smaller remaining portion at the end if necessary

It updates the file position after reading. It also checks for EOF and returns only as many bytes as available.

b_seek(int fd, off_t offset, int whence)

This lets the user change the current file position. Depending on the whence value (beginning, current, or end), it calculates the new position. It validates that the position is within file bounds and resets internal tracking so future reads or writes start from the right spot.

b_close(int fd)

When we're done using a file, we call this to free the memory buffer used by the file and clear its control block in fcbArray. This makes the slot reusable for another file later.

initFreeSpaceManager(FreeSpaceManager *fsm, VCB *vcb, uint32_t blkSize)

This routine establishes and persists your volume's free-block bitmap at system startup. It first validates its inputs, then records the total number of blocks and block size from the VCB so subsequent allocators know the layout. It computes exactly how many bytes are needed to track every block (rounding up bits->bytes), then further rounds that to whole blocks so it can read/write the bitmap in LBA-sized chunks. After allocating a zeroed buffer in memory, it marks every block below freeBlockStart (the VCB, bitmap blocks themselves, root directory, etc.) as "used" by setting those bits. Finally, it immediately writes the bitmap out to disk, ensuring your on-disk free-space map always reflects that reserved region before any allocations occur.

allocateBlocks(FreeSpaceManager *fsm, uint32_t count, uint32_t, allocatedBlocks)

This function implements first-fit allocation of N consecutive blocks. It scans the bitmap bit by bit, counting how many free blocks in a row (consec) it's seen, resetting on a used bit. When consec reaches the requested count, it calls mark_run_used on that exact run (atomically allocating them), then immediately writes the updated bitmap back to disk so on-disk state remains in sync. If no contiguous span large enough exists, it returns -1 to signal "out of space." This behavior ensures your file data always resides in one run and that the free-space map never drifts out of date.

releaseBlocks(FreeSpaceManager *fsm, uint32_t *blockList, uint32_t count)

When a file or directory is deleted or shrunk, this function frees exactly the blocks the caller previously allocated. Rather than scanning the entire bitmap, it iterates only the provided block list, clearing each bit to mark blocks as free. It guards against invalid block numbers by skipping any out-of-range entries. After unsetting all bits, it writes the refreshed bitmap to disk so the newly freed space is immediately available to future allocations, preserving consistency between memory and disk.

alloc_one_block(FreeSpaceManager *fsm)

A trivial convenience wrapper around allocateBlocks for the very common case of allocating exactly one block. It invokes the general allocator with a count of one and returns either the newly-allocated block number or UINT32_MAX on failure. By encapsulating the single-block case, callers avoid boilerplate arrays and error handling, making the most frequent allocation path both simpler and less error-prone.

mark_run_used(FreeSpaceManager *fsm, uint32_t start, uint32_t len, uint32_t *out)

Helper marks a contiguous span of blocks as allocated in the in-memory bitmap. Given a start index and length, it loops exactly over that range (to guarantee contiguity), sets each corresponding bit, and—if requested—records the actual block numbers into the caller's array. By centralizing bit-setting logic, higher-level allocators avoid reimplementing the same loop and bit-math on every allocation, ensuring consistency and reducing the risk of off-by-one errors.

initFileSystem (uint64_t numberOfBlocks, uint64_t blockSize)

This function is the orchestrator for bringing the file system from raw disk blocks into a usable, directory-based environment. On the first call, it allocates one block's worth of memory for the Volume Control Block (VCB) so all volume metadata will fit in exactly one LBA. It then reads that first block from disk and checks the VCB signature to determine whether the volume was previously formatted. If not, it zeroes the VCB structure, fills in identifying fields (volume name, signature, block size, total block count), computes where the free-space bitmap must live (immediately after the VCB), and initializes the free-space manager to build and persist that bitmap. Next, it creates your root directory in a newly allocated run of blocks (via createDir), records its starting block in the VCB, and updates the free-block count to reflect everything reserved so far (VCB, bitmap, root directory). Finally, it writes the updated VCB and bitmap back to disk, cleans up temporary memory, and prints confirmation. On subsequent calls when the VCB signature is already present, it simply reinitializes the in-memory free-space manager from the on-disk bitmap, readying the system for directory and file operations without reformatting.

exitFileSystem ()

This cleanup function gracefully tears down your file system when the shell exits. It prints a final message, then frees the heap-allocated VCB block (if any), setting the global pointer to NULL to avoid dangling references. By releasing the VCB's memory, you prevent leaks and ensure that any further operations on the in-memory structures are safely detected as invalid, guarding against accidental use after shutdown.

createDir(int initialNumEntries, directory_entry* parent, FreeSpaceManager* fsm)

This function encapsulates everything needed to bring a brand-new directory into existence on the volume. Given a desired number of entries, it computes how many blocks that directory will occupy (rounding up to the block boundary), allocates an in-RAM buffer of exactly that size, then reserves a contiguous run of on-disk blocks via the free-space manager. It stamps out the mandatory "." self-reference and ".." parent entries, copying parent metadata or, in the root's case, pointing back at itself—tagging each with creation/modification/access timestamps, owner IDs, a unique file number, and marking them as in-use. Finally, it persists the new directory data to its allocated LBAs and writes back any changes to the free-space bitmap so the allocation is durable. By returning the in-memory directory buffer (with its block list still held in the caller's context), higher layers can immediately incorporate this new directory's location into the VCB or parent directory structure.

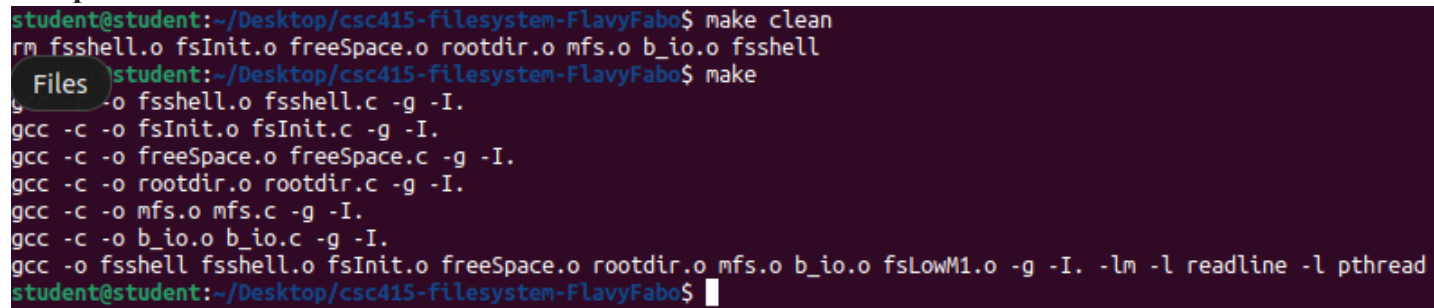
writeDir(directory_entry* parent, int blocksNeeded, int startBlock)

As a thin middleware over LBAwrite, writeDir provides a single, centralized entry point for flushing any in-memory directory table back to its on-disk blocks. Added to add on to and add more error checking, but didnt end up completing this as wasnt needed for short time period for assignment.

A detailed table of who worked on what, down to the individual function level:

Name	What Part worked on
Arric Sekhon	In milestone 1, I worked on VCB, fsinit.c. In milestone 2, I worked on fs_setcwd, fs_getcwd, fs_isFile, fs_isDir, fs_mkdir fs_opendir, fs_readdir, fs_closedir. In milestone 3, I worked on the b_open function.
Aidan Bayer-Calvert	I worked on the free space and fsinit in milestone 1. In milestone 2/3 I was in charge of fs_rmdir, fs_delete, fs_stat, cmd_cp, cmd_mv, and cmd_rm. I also worked a lot on formatting and commenting all the code as well as debugging and fix errors across the whole project.
Fabian Camarena Veliz	In milestone 1, I was in charge of building the root directory. Built 2 functions: createDir() and writeDir(). In milestone 2, I helped out with split_path() which is our version of parsePath(). In milestone 3 I helped out with b_open, b_read, and ls
Subhan Khan	In milestone 1 worked on free space class. In milestone 2 and milestone 3 worked on getting the implementations of commands cp, cp2l, and cp2fs working.

Compiles:

A terminal window with a dark purple background. The prompt is 'student@student:~/Desktop/csc415-filesystem-FlavyFabo\$'. The first command is 'make clean', followed by 'rm fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsshell'. Then, a green 'Files' icon is shown. The next command is 'make', followed by several 'gcc -c' commands for individual source files: 'fsshell.c', 'fsInit.c', 'freeSpace.c', 'rootdir.c', 'mfs.c', and 'b_io.c'. The final command is 'gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1.o -g -I. -lm -l readline -l pthread'. The prompt returns to 'student@student:~/Desktop/csc415-filesystem-FlavyFabo\$' with a cursor.

```
student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make clean
rm fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsshell
Files student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1.o -g -I. -lm -l readline -l pthread
student@student:~/Desktop/csc415-filesystem-FlavyFabo$
```

Screen shots showing each of the commands listed in the readme:

IMPORTANT NOTES:

- some screenshots show multiple commands, the remaining commands shown in each screenshot are listed above the image
- The err=2 in the screenshots appears because we recreate the SampleVolume on each run. Doing so lets us demonstrate how the commands behave on a fresh, empty volume, making their effects as clear as possible.

md - Make a new directory

touch - creates a file

ls - Lists the file in a directory

```

student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1.o -g
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status - |
| ls                      |      ON      |
| cd                      |      ON      |
| md                      |      ON      |
| pwd                    |      ON      |
| touch                  |      ON      |
| cat                    |      ON      |
| rm                     |      ON      |
| cp                     |      ON      |
| mv                     |      ON      |
| cp2fs                  |      ON      |
| cp2l                   |      ON      |
|-----|
Prompt > md directory_1
Prompt > touch file1
Prompt > ls

directory_1
file1
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$

```

mv - Moves a file - source dest

cd - Changes directory

```

student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o
./fsshell SampleVolume 100000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status -|
| ls                      |      ON      |
| cd                      |      ON      |
| md                      |      ON      |
| pwd                    |      ON      |
| touch                  |      ON      |
| cat                    |      ON      |
| rm                     |      ON      |
| cp                     |      ON      |
| mv                     |      ON      |
| cp2fs                  |      ON      |
| cp2l                   |      ON      |
|-----|
Prompt > touch file1
Prompt > md dir
Prompt > ls

file1
dir
Prompt > mv file1 dir
Prompt > ls

dir
Prompt > cd dir
Prompt > ls

file1
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$

```

cp - Copies a file - source [dest]


```

student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| Status -|
| ls                      | ON   |
| cd                      | ON   |
| md                      | ON   |
| pwd                    | ON   |
| touch                  | ON   |
| cat                    | ON   |
| rm                     | ON   |
| cp                     | ON   |
| mv                     | ON   |
| cp2fs                  | ON   |
| cp2l                   | ON   |
|-----|
Prompt > touch file1
Prompt > cp file1 newfile
Prompt > ls

file1
newfile
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$

```

rm - Removes a file or directory

```

student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsL
./fsshell SampleVolume 100000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 byt
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status -|
| ls                      |      ON      |
| cd                      |      ON      |
| md                      |      ON      |
| pwd                    |      ON      |
| touch                  |      ON      |
| cat                    |      ON      |
| rm                     |      ON      |
| cp                     |      ON      |
| mv                     |      ON      |
| cp2fs                  |      ON      |
| cp2l                   |      ON      |
|-----|
Prompt > touch file1
Prompt > md directory1
Prompt > ls

file1
directory1
Prompt > rm file1
Prompt > rm directory1
Prompt > ls

Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$

```

cat - (limited functionality) displays the contents of a file
cp2fs - Copies a file from the Linux file system to the test file system

cp2l - Copies a file from the test file system to the linux file system

```
student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status -|
| ls                      |      ON  |
| cd                      |      ON  |
| md                      |      ON  |
| pwd                    |      ON  |
| touch                  |      ON  |
| cat                    |      ON  |
| rm                     |      ON  |
| cp                     |      ON  |
| mv                     |      ON  |
| cp2fs                  |      ON  |
| cp2l                   |      ON  |
|-----|
Prompt > cp2fs test.txt
Prompt > ls

test.txt
Prompt > cat test.txt
Lorem ipsum
Prompt > cp2l test.txt newfile.txt
Prompt > ls

test.txt
Prompt > cp2fs newfile.txt
Prompt > ls

test.txt
newfile.txt
Prompt > cat newfile.txt
Lorem ipsum
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$
```

pwd - Prints the working directory

```

student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status -|
| ls                      |      ON  |
| cd                      |      ON  |
| md                      |      ON  |
| pwd                    |      ON  |
| touch                  |      ON  |
| cat                    |      ON  |
| rm                     |      ON  |
| cp                     |      ON  |
| mv                     |      ON  |
| cp2fs                  |      ON  |
| cp2l                   |      ON  |
|-----|
Prompt > pwd
/
Prompt > md hi
Prompt > cd hi
Prompt > pwd
/hi
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$

```

history - Prints out the history

```
student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| - Status -|
| ls                      |      ON      |
| cd                      |      ON      |
| md                      |      ON      |
| pwd                    |      ON      |
| touch                  |      ON      |
| cat                    |      ON      |
| rm                     |      ON      |
| cp                     |      ON      |
| mv                     |      ON      |
| cp2fs                  |      ON      |
| cp2l                   |      ON      |
|-----|
Prompt > md directory
Prompt > ls

directory
Prompt > history
md directory
ls
history
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$
```

help - Prints out help

```
student@student:~/Desktop/csc415-filesystem-FlavyFabo$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o freeSpace.o freeSpace.c -g -I.
gcc -c -o rootdir.o rootdir.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -o fsshell fsshell.o fsInit.o freeSpace.o rootdir.o mfs.o b_io.o fsLowM1
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Volume not formatted. Formatting now...
Root directory initialized. VCB updated and written to disk.
|-----|
|----- Command -----| Status |
| ls                      |      ON      |
| cd                      |      ON      |
| md                      |      ON      |
| pwd                    |      ON      |
| touch                  |      ON      |
| cat                    |      ON      |
| rm                     |      ON      |
| cp                     |      ON      |
| mv                     |      ON      |
| cp2fs                  |      ON      |
| cp2l                   |      ON      |
|-----|
Prompt > help
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt > exit
System exiting....
student@student:~/Desktop/csc415-filesystem-FlavyFabo$
```